

Modül 05: Sınıflar ve Nesneler

Bu mezuniyet günü. Şimdiye kadarki her dosya kompakt kaynak dosyasıydı. Buradan itibaren örnekler klasik biçimde yazılıyor, çünkü her gerçek kod tabanı, her öğretici ve şimdiye kadar yazılmış her Stack Overflow cevabı onu kullanıyor.

Dilde yeni olan hiçbir şey yok. Modül 01, derleyicinin sınıfı baştan beri sizin için ürettiğini `javap` ile gösterdi. Şimdi onu kendiniz yazıyorsunuz.

Neler Öğreneceksiniz

- `public static void main(String[] args)` içindeki her kelime ve her birinin nedeni
- Alanlar, dört görünürlük seviyesi ve neden `private` ile başladığınız
- Kurucular, `this(...)` ve bir tane yazınca kaybettiğiniz bedava kurucu
- Statik fabrika metotlarının kuruculardan neden sıklıkla üstün olduğu
- `super()` öncesinde doğrulama yapmanızı sağlayan esnek kurucu gövdeleri

Başka Bir Dilden Geliyorsanız

	Python	Java
Kurucu	<code>__init__</code>	sınıfla aynı adlı metot
Mevcut örnek	açık <code>self</code> parametresi	örtük <code>this</code>
Gizlilik	gelenek olarak <code>_ad</code>	derleyicinin uyguladığı <code>private</code>
Birden fazla kurucu	tek <code>__init__</code> , varsayılan argümanlar	aşırı yükleme ya da statik fabrikalar
Sınıf düzeyinde veri	sınıf niteliği	<code>static</code> alan
Dosya düzeni	dosya başına serbestçe birçok sınıf	dosya başına bir <code>public</code> sınıf, adı eşleşmeli

Asıl fark gizlilik satırı. Python'un alt çizgisi bir ricadır. Java'nın `private` niteleyicisi derleyicinin uyguladığı bir kuraldır ve onu ihlal eden kod derlenmez.

Ders

Klasik biçim, kelime kelime

```
public class TheClassicForm {  
    public static void main(String[] args) {  
        System.out.println("Running the classic way.");  
    }  
}
```

Kelime	Nedeni
public (sınıf)	her yerden kullanılabilir. Dosya adıyla eşleşmeli.
public (main)	JVM'in onu çağırabilmesi gerekiyor
static	asıl önemli olan. Başlangıçta henüz hiçbir nesne yok, dolayısıyla main bir nesneye ait olamaz.
void	hiçbir şey döndürmez; çıkış kodları System.exit ile gelir
main	JVM'in aradığı tam ad
String[] args	komut satırı argümanları. Asla null değil; yoksa boş.

main içinde this yoktur, çünkü statiktir. Örnek durumunu kullanmak için main önce bir nesne kurar, ki gerçek main metodlarının çoğu bunu yapar.

Hangi biçimi kullanmalısınız? Öğrenmek, script'ler ve tek dosyalık araçlar için kompakt biçim daha az gürültülü. Çok dosyalı herhangi bir şey, bir kütüphane ya da başkalarının dokunduğu kod için klasik biçimi kullanın. Aynı dildir; ne kadar yazacağınızı seçiyorsunuz.

Kapsülleme

Dört görünürlük seviyesi, en dardan başlayarak:

Seviye	Erişim
private	yalnızca bu sınıf
(hiçbiri)	bu paket. Paket-özel denir.
protected	bu paket, artı her yerdeki alt sınıflar
public	herkes

private ile başlayın ve sebebiniz olduğunda genişletin. Sonradan genişletmek kolaydır. Sonradan daraltmak ona bağımlı olan herkesi kırar.

Amaç gizlilik değil, **değişmezlerdir**. [Encapsulation.java](#) dosyasından:

```
refused: insufficient funds: have 15000, asked for 100000  
balance is still: 15000
```

`balanceInCents` alanı `private` olduğu için her değişiklik kuralları uygulayan bir metottan geçer, böylece sınıf bakiyenin asla negatif olmayacağına söz verebilir ve sözünü gerçekten tutabilir. `Public` değiştirilebilir alanları olan bir sınıfın hiçbir değişmezi yoktur, çünkü herhangi bir şey onları her an değiştirebilir.

Alanda `final` tam olarak bir kez, kurucuda atanır demektir. Dikkat: bu değişmez olmakla aynı şey değildir, `final` bir alan hâlâ değiştirilebilir bir nesneyi gösterebilir.

Kurucular

Herhangi bir kurucu yazmak bedava olanı kaldırır. Java argümansız kurucuyu yalnızca hiç kurucu bildirmediğinizde sağlar. Mevcut bir sınıfa kurucu eklemenin `new Thing()` yazan çağırırları kırabilmesinin sebebi budur.

`this(...)` başka bir kurucuya devreder:

```
Temperature() {  
    this(0.0);  
}
```

Statik fabrikalar kurucuları sıklıkla yener çünkü adları vardır:

```
Temperature.fromFahrenheit(77.0)  
Temperature.fromKelvin(298.15)
```

Tek bir `double` alan iki kurucu birlikte var olamaz. İki fabrika olabilir. Bir fabrika ayrıca ön belleklenmiş bir örnek ya da bir alt sınıf döndürebilir, kurucu ise her zaman yeni bir şey kurar.

Alanlar varsayılan alır, yereller almaz. Bir alan otomatik olarak `0`, `false`, `null` ya da `0.0` olur. Atanmadan kullanılan yerel değişken derleme hatasıdır. Bu asimetri bilinçlidir: bir alan meşru olarak başka bir metot tarafından sonradan atanabilir, dolayısıyla derleyici hiçbir şey kanıtlayamaz; yerel değişken ise bildirildiği yerden birkaç satır ötede kullanılır, dolayısıyla derleyici denetleyebilir ve denetler.

Esnek kurucu gövdeleri

Eski kural: `super(...)` ilk ifade olmak zorundaydı. Üst nesneyi kurar, sonra argümanların geçersiz olduğunu keşfederdiniz.

JDK 25'te kesinleşen JEP 513, önce ifade çalıştırmaya izin veriyor:

```
Positive(int value) {  
    if (value <= 0) {  
        throw new IllegalArgumentException("must be positive, got " + value);  
    }  
    super(value);  
}
```

`FlexibleConstructorBodies.java` çıktısı:

```
Measurement constructor ran with 5
built: 5

now with an invalid value:
rejected: must be positive, got -1
```

Dikkatle okuyun. Üst sınıf kurucusu geçerli durum için **bir kez** çalıştı. Geçersiz olan için hiç çalışmadı, çünkü denetim önce fırlattı. Eski kural altında bu, denetimi `super()` çağrısının içindeki statik bir yardımcıya gizlemeden imkânsızdı.

`super()` öncesinde `this` hâlâ yasaktır, çünkü üst sınıf çalışmamıştır ve nesne henüz geçerli değildir. Derleyici bunu uygular.

Çalıştırın

```
java modules/05-classes-and-objects/examples/TheClassicForm.java
java modules/05-classes-and-objects/examples/TheClassicForm.java hello world
java modules/05-classes-and-objects/examples/Encapsulation.java
java modules/05-classes-and-objects/examples/Constructors.java
java modules/05-classes-and-objects/examples/FlexibleConstructorBodies.java

./scripts/verify-examples.sh modules/05-classes-and-objects
```

Sık Yapılan Hatalar

Public değiştirilebilir alanlar. Sınıf artık kendi durumu hakkında hiçbir söz veremez.

Kurucu ekleyip başka yerdeki `new Thing()` çağrılarını kırmak. Herhangi bir kurucu bildirdiğiniz anda bedava argümansız kurucu kaybolur.

Getter'dan iç koleksiyonunuzu döndürmek. Çağırان artık nesnenizin durumunu sizden geçmeden değiştirebilir. Modül 04'ün ödevi çözümü kapsıyordu.

`final` kelimesinin değişmez demek olduğunu sanmak.

```
private final List<String> items = new ArrayList<>();
items.add("still allowed"); // fine, the reference never changed
```

`final` referansı sabitler, nesneyi değil.

Gerçek iş yapan bir kurucu. Dosya açan ya da ağ çağrısı yapan kurucuların testi zordur ve çağırının başa çıkamayacağı biçimlerde başarısız olurlar. Nesneyi kurun, sonra bir metod çağırın.

Her alan için refleksle getter ve setter yazmak. Her alan için setter, fazladan adımlı bir public değiştirilebilir alandır. Gerçekten bir şeyin o değeri dışarıdan değiştirmesi gerektiğinde ekleyin.

Özet Çıkarımlar

- `main` üzerindeki `static` , başlangıçta hiçbir nesne olmadığı için vardır.
- **Klasik ve kompakt biçim aynı dildir**, yalnızca yazdığınız şey farklıdır. Çok dosyalı ya da paylaşılan her şey için klasik olanı kullanın.
- **Alanlara `private` ile başlayın.** Genişletmek kolay, daraltmak çağırıcıları kırar.
- **Kapsülleme size gizlilik değil değişmez satın alır.**
- **Herhangi bir kurucu bildirmek bedava argümansız olanı kaldırır.**
- **Statik fabrikaların adı vardır**, dolayısıyla kuruculardan farklı olarak aşırı yüklenebilirler.
- **Alanlar varsayılan alır, yereller almaz.**
- **İfadeler artık `super()` öncesinde gelebilir**, böylece kötü argümanları üst nesne hiç kurulmadan reddedebilirsiniz.

Ödev

[homework/README.md](#)

Geçersiz duruma sokulamayan bir sınıf kurun, sonra deneyerek kanıtlayın. Referans çözüm

[solutions/05-classes-and-objects/](#) altında.